

Reviewer Pack — Minimal Rank of Tropicalized Hodge Decomposition Bounds BP R...

Entry 389197748758 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 07:14:24 UTC

Minimal Rank of Tropicalized Hodge Decomposition Bounds BP Read-Twice Size

Entry ID: 389197748758 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 07:14:24 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory: BP Read-Twice Complexity

Statement:

[‘For a given read-twice branching program P , define the minimal rank of its tropicalized Hodge decomposition as $\rho_H(P)$ and the size of P as $s(P)$. Then, $\rho_H(P) = O(\log(s(P)))$ but $\rho_H(\text{IP}_2 \text{ trivial BP}) = \Omega(n)$, where IP_2 is the inner product mod 2 problem.’, ‘For every instance of IP_2 with n variables, there exists a read-twice BP P such that $s(P) \leq c^n$ for some constant c and $\rho_H(P) \leq \log(s(P)) + O(1)$.’]

Rationale (proposer’s reasoning):

[‘The Hodge decomposition is a powerful tool in algebraic geometry that provides a way to understand the structure of complex varieties. If the minimal rank of the tropicalized Hodge decomposition of read-twice BPs is related to their size, it could offer insights into the complexity of solving problems like IP_2 using read-twice BPs.’, ‘This connection between algebraic geometry and computational complexity theory has not been extensively explored before, suggesting a potential new avenue for research.’]

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 59e560cdc165a3f5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the minimal rank of tropicalized Hodge decomposition $\rho_H(P)$, if the ratio $|\rho_H(P) - \log(s(P))|$ is ≤ 0.1 for all seeds and the mean $\rho_H(P)$ across seeds is $\leq \log(s(P)) + 1$, then support is provided; otherwise, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropicalized Hodge decomposition" AND "BP Read-Twice Complexity" - "minimal rank" AND "Hodge theory" AND "read-twice branching program" - "IP_2 problem" AND "size of BP" AND "tropicalization"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def inner_product_mod_2(x, y):
        return sum(int(a) * int(b) % 2 for a, b in zip(x, y))

    def trivial_bp(n):
        return [[inner_product_mod_2(bin(i)[2:].zfill(n), bin(j)[2:].zfill(n)) for j in range(1 <= n)] for i in range(1 <= n)]

    def tropicalized_hodge_decomposition(P):
        # Simplified version of the Hodge decomposition
        return sum(sum(row) for row in P)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        s_P = len(trivial_bp(n)) ** 2
        rho_H_P = tropicalized_hodge_decomposition(trivial_bp(n))
        results.append({
            "n": n,
            "s_P": s_P,
            "rho_H_P": rho_H_P,
            "ratio": abs(rho_H_P - math.log(s_P)),
        })

    mean_ratio = sum(result["ratio"] for result in results) / len(results)
```

```

max_rho_H_P = max(result["rho_H_P"] for result in results)
support_fraction = all(result["ratio"] <= 0.1 for result in results)

return {
    "metric_name": "Ratio  $|\rho_H(P) - \log(s(P))|$ ",
    "metric_value": mean_ratio,
    "instances_tested": len(results),
    "conjecture_holds": support_fraction and max_rho_H_P <= math.log(max([result["s_P"] for result in results]))
    "counterexample": "" if support_fraction else f"max_rho_H_P={max_rho_H_P} > log(s(P)) + 1"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_ratio = sum(result["metric_value"] for result in results) / len(results)
    max_rho_H_P = max(result["counterexample"].split("=")[1].strip() if result["conjecture_holds"] else 0 for result in results)
    support_fraction = all("support" in result["counterexample"] for result in results)

    if support_fraction:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=NA support_fraction={support_fraction}")
    elif max_rho_H_P > 0:
        print(f"RESULT: FALSIFIED counterexample='max_rho_H_P={max_rho_H_P} > log(s(P)) + 1' first")
    else:
        print(f"RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, making it impossible to evaluate the conjecture's conditions. | next: Re-run the test with a longer timeout or modify the code to ensure it completes within the time limit.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12317
2	preregistration	ollama_remote	glm4:latest	0	0	6085
3	novelty	ollama_remote	glm4:latest	0	0	4764
4	novelty	ollama_remote	glm4:latest	0	0	5736
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13559
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7719
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13208
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8587
9	judge	ollama_remote	glm4:latest	0	0	8202

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 80176 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/389197748758.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/389197748758.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/389197748758.tar.gz` (if generated)